Data Scientist Technical Challenge – Mercado Livre

Candidate: Guilherme Butzke Schreiber Gering
March 2026

Context

This report was built to answer questions for the Mercado Libre challenge by the candidate Guilherme Butzke. It follows the same sequence as the code construction developed in this repository: https://github.com/guilherme-butzke/ps-meli

The framework was divided into four steps:

 # Phase 00 - Strategic Modeling Framework
 # Phase 01: Exploratory Data Analysis (EDA)
 # Phase 02: Feature Engineering & Target Analysis
 # Phase 03: Model Selection & Profit Optimization

Each of these steps is explained and documented in this PDF, and the complete code, with the same names, is available in the **notebooks** folder in the repository

Phase 00 - Strategic Modeling Framework

To model this problem, we must shift from standard accuracy-based metrics to a **Financial Cost Function**. The goal is to find the threshold where the cost of a "False Alarm" (blocking a good customer) equals the cost of a "Missed Fraud" (allowing a thief to transact).

The goal of this phase is to define the profit objective function to optimize the profit and create a framework to solve this problem as a ML approach.

Phase 00 - Strategic Modeling Framework

1. The Profit Objective Function

The goal of this analysis is:

"Improve profit by reducing loss using a model able to detect fraud."

Conversely, True Positives (TP) and True Negatives (TN) do not actively generate *new* profit from the perspective of the model's action. As a guiding principle:

"Preventing a loss is not the same as generating revenue."

For fraud prevention, maximizing the company's profits does not mean being profitable, but reducing losses. So, we considerer only False Negatives (FN) and False Positives (FP) to maximize the follow metric:

$$\boldsymbol{M = -(1.0\,.FN) - (0.25\,.FP)}$$

More details and math proof are present in notebook `*00_strategic_modeling_framework.ipynb*`

Phase 00 - Strategic Modeling Framework

2. Development Strategy

To construct this model to predict fraudulent transactions, we followed an iterative approach close to the CRISP methodology. Each notebook documents specific actions, analyzes results, and refines the strategy for the subsequent phase.

In the *notebooks* folder, each notebook file, beyond the code used to generate the results, documents specific actions, analyzes results, and refines the strategy for the subsequent phase.

These are the steps:

Phase 1: Exploratory Data Analysis (EDA): Dataset Description, Insight Extraction, and Feature Selection
Phase 2: Feature Engineering & Target Analysis: Literature Review, Feature Prioritization, Variable Relevance, and Hypothesis Formulation
Phase 3: Model Selection & Profit Optimization: Algorithm Training, Cost-Sensitive Learning, and Threshold Optimization

At the conclusion of each phase, the strategy will be updated based on data findings. The final output will be a production-ready ML model optimized for ****Maximum Net Profit****.

Phase 01: Exploratory Data Analysis (EDA)

To accelerate the generation of the initial report, we used Pandas Profiling, which automatically produces descriptive statistics and relevant insights about the variables. After this first assessment, we were able to identify important patterns in the data and generate an initial list of insights on how each feature should be treated during preprocessing and feature engineering.

Overview

Overview | Alerts 22 | Reproduction

Dataset statistics

Number of variables	19
Number of observations	150000
Missing cells	135771
Missing cells (%)	4.8%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	54.1 MiB
Average record size in memory	377.9 B

Variable types

Categorical	3
Numeric	11
Text	2
Boolean	2
DateTime	1

Pandas profiling screenshot

Phase 01: Exploratory Data Analysis (EDA)

With **Pandas Profiling**, we generated:

- an **HTML report** with a detailed overview of the data
- two **CSV files** with the main descriptive highlights and suggestions of variables to keep or drop

All of these files are saved in notebooks/results. The table below shows the justification for these decisions.

a	Cat	85.7% concentrated in one category	Keep	Minority categories may carry higher
b	Numeric	8.7% missing	Keep	Missingness may be predictive
c	Numeric	8.7% missing	Keep	Same reasoning as b
d	Numeric	Correlated with m	Keep	Potential predictive signal
e	Numeric	43.4% zeros, extreme skew	Keep	Strong behavioral signal candidate
f	Numeric	16.9% zeros, highly skewed	Keep	Tail behavior likely informative
g	Cat	Country-like variable	Keep	Geographic fraud patterns common
h	Numeric	8.6% zeros	Keep	Zero may indicate inactivity
j	Cat	Low cardinality (5.5% distinct values)	Keep	Few categories may carry fraud signa
k	Numeric	100% unique	Drop	Identifier; leakage risk
l	Numeric	Correlated with f	Keep	Potential predictive contribution
m	Numeric	10.6% zeros, correlated with d	Keep	Behavioral indicator
n	Cat	Binary-looking	Keep	Model-ready
o	Numeric	72.6% missing	Keep	Missingness likely predictive
p	Cat	Binary categorical	Keep	Risk segmentation feature
fecha	DateTime	Timestamp	Keep	Temporal fraud patterns
monto	Numeric	Heavy-tailed distribution	Keep	Core economic driver
score	Numeric	Pre-existing score	Keep	Likely strong predictor (check leakag
fraude	Binary	5% positive	Keep	Target

Phase 02: Feature Engineering & Target Analysis

The work done in this phase was:

- **Technical Literature Review:** Researched and selected specific preprocessing techniques (e.g., handling extreme skewness, high cardinality, and missingness) identified during the Phase 1 exploration.
- **Feature Prioritization:** Compiled a list of potential new variables and ranked them by their expected predictive power and implementation complexity.
- **Feature Engineering:** Derived new features based on the initial hypotheses (such as temporal extractions from fecha or behavioral ratios) to enhance the model's predictive performance.
- **Target Characterization:** Analyzed the distribution of the fraude variable and its specific relationship with other variables.
- **Variable Relevance:** Used statistical tests (e.g., Chi-Squared for categorical data, ANOVA for numerical data) to quantify the relationship between features and the target.
- **Hypothesis Formulation:** Created a list of "Risk Indicators" (e.g., "Transactions above amount X are $Y%$ more likely to be fraud").
- **Feature Selection:** If necessary, additional feature selection was performed after these previous steps.

Phase 02: Feature Engineering & Target Analysis

Technical Literature Review: The following table summarizes the proposed solutions aligned with best practices from the literature review done and how solve each problem.

Problem Identified	Variables Affected	Technical Risk	Proposed Solutions	Business Justification
Target imbalance (5% fraud)	fraude	Model biased toward majority class; accuracy misleading	Cost-sensitive learning; class weights; threshold optimization based on profit; PR-AUC; Expected Profit curve	Fraud cost is asymmetric (−100% vs +25%)
High missingness (72.6%)	o	Imputation may hide signal	Missing indicator; missing as separate bin (WOE); compare imputation strategies; tree models with native missing handling	Missingness likely behavioral
Moderate missingness (8.7%)	b, c	Bias or instability	Missing indicators; median imputation; WOE bin with missing category	Incomplete information may correlate with fraud
Unique identifier (100% unique)	k	Leakage / overfitting	Drop variable	No predictive generalization
High zero concentration	e, f, m, h	Structural zero vs true zero unclear	Zero indicators; separate bin for zero; two-part modeling; log1p transformation	Zero may represent absence of activity
Extreme skewness (very high γ1)	e, f	Linear instability; tail dominance	Log transform; Box–Cox; Yeo–Johnson; quantile transformation; WOE binning; percentile bins	Stabilizes model; captures non-linear fraud risk
Heavy tail in monetary variable	monto	Extreme values dominate loss	Log transform; Box–Cox / Yeo–Johnson; percentile binning; top-value flag; interaction terms	High-value fraud drives economic loss
Correlation clusters	(d, m), (e, monto), (f, l)	Multicollinearity in LR	VIF analysis; drop redundancy; optional PCA; tree models handle naturally	Avoid unstable coefficients
Categorical imbalance (85.7% one category)	a	Rare categories unstable	Rare grouping; WOE encoding; frequency encoding	Minority categories may carry higher fraud rate
High cardinality (~8,250 distinct values)	j	One-hot infeasible; sparse representation; overfitting	Rare grouping; frequency encoding; target encoding (smoothed); hashing trick	High-cardinality categories may concentrate fraud in specific values
Temporal variable unused	fecha	Loss of behavioral signal	Extract hour, weekday; cyclic encoding	Fraud often temporal
Behavior–amount interaction ignored	e, monto	Interaction effect missed	Ratio feature; interaction terms	Abnormal intensity patterns signal fraud
Outliers removal risk	e, f, monto	Removing fraud signal	Avoid trimming; prefer transformation; robust scaling	Fraud often lives in tails
Dataset clean (no duplicates)	Dataset	No issue	No action required	Data integrity confirmed

Phase 02: Feature Engineering & Target Analysis

Feature Prioritization and Engineering:

- At this stage, we created a list of features that could potentially be useful for modeling.
- Not all of them will necessarily be useful, but since this is still an exploratory phase, we can analyze different options.
- The goal here is to create features and then choose the best ones according to their correlation with the target or according to their strength within the model.

As a result, **43 new features were created**. Some examples of the created features are:

- **b_is_missing** (Missingness may represent abnormal behavior)
- **e_is_zero** (Checks whether the presence of zero values is relevant to the target)
- **d_top1_flag** (Tail anomaly detection)
- **bin_c** (Binned variable, which is very common in finance and credit score analysis)
- **rare_group_a** (Stabilizes sparse categories)
- **interaction_monto_e** (Ratio between a and e to reduce skewness and capture non-linear relationships)
- **boxcox_e / log_e / yeojohnson_e** (Reduce skewness)
- **PCA_dmc** (PCA between variables)

Phase 02: Feature Engineering & Target Analysis

Target Characterization
After generating all these features, we will create and analyze the relationship between each variable (or combinations of them) and the main target.

For this purpose, we created a set of statistical tests to evaluate the relationship between the target and each variable.

- **Continuous variables**: KS Statistic (Kolmogorov–Smirnov), Mann–Whitney U Test, Point-Biserial Correlation, Cohen’s d, and ROC-AUC.
- **Categorical variables**: Chi-Squared Test, Cramér's V ,and Information Value (IV) with Weight of Evidence (WoE).

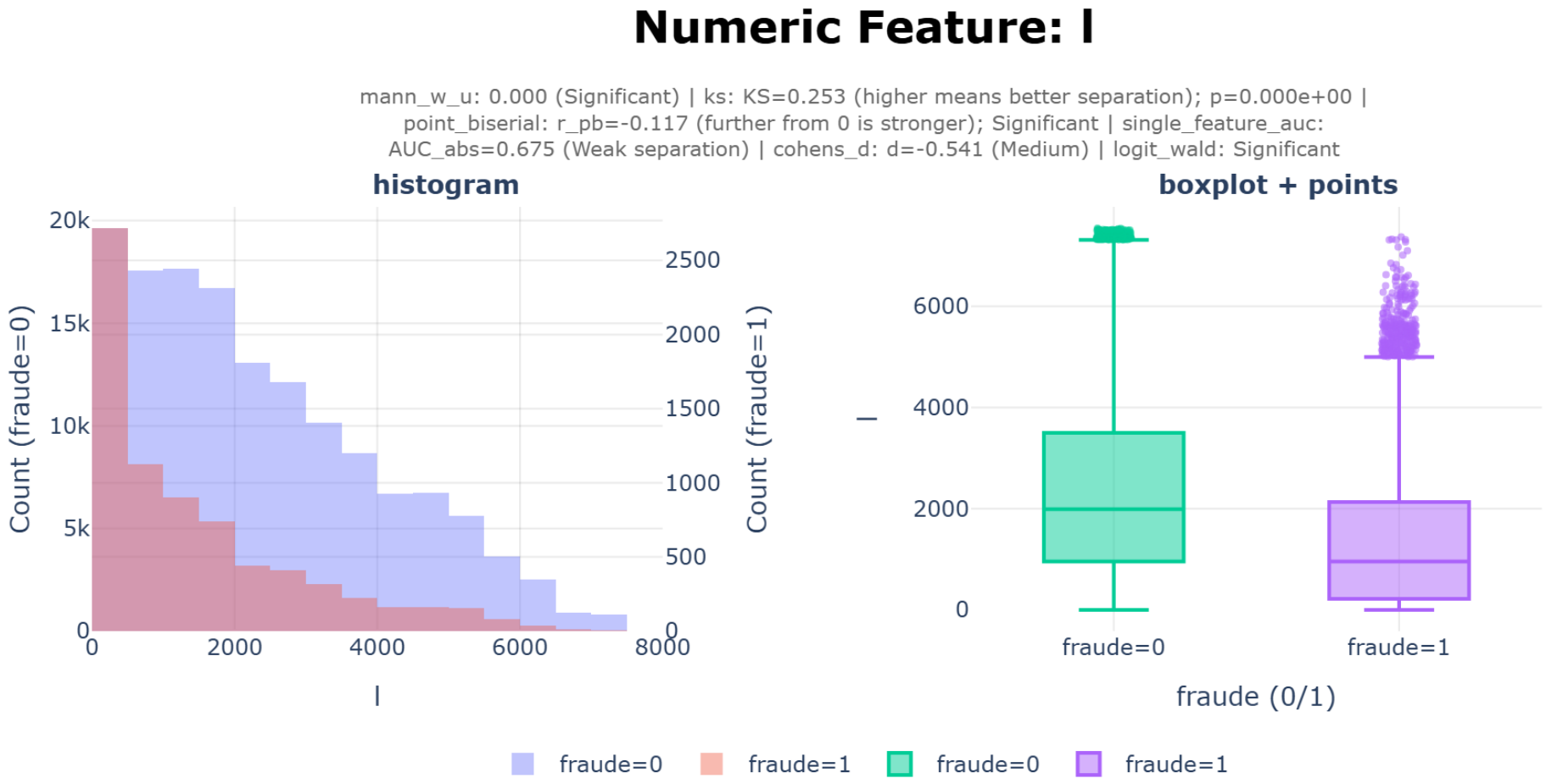
Additionally, to support the analysis, we plotted graphs showing the behavior of each variable against the fraud target.

All images for each variable (both numerical and categorical) were saved in the folder *results/images*. Each image also includes the values of the calculated metrics.

Phase 02: Feature Engineering & Target Analysis

Target Characterization

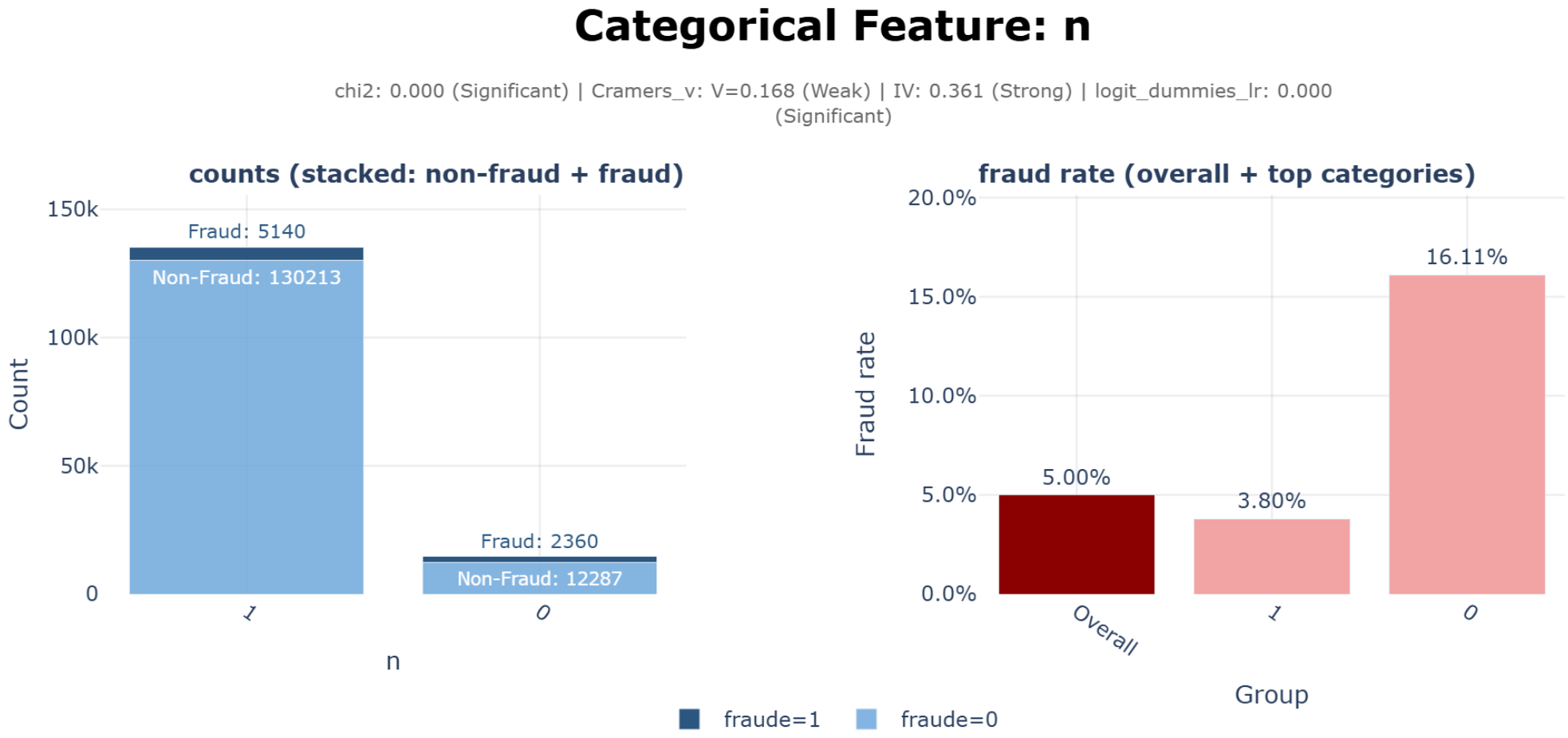
Example of image generated after target characterization with numerical feature



Phase 02: Feature Engineering & Target Analysis

Target Characterization

Example of image generated after target characterization with categorical feature



Phase 02: Feature Engineering & Target Analysis

Target Characterization - General Results – Continuos Variables

- *Transformed vs. Raw Features*: Features with transformations to reduce skew like ``e``, ``log_e``, ``boxcox_e``, and ``yeojohnson_e`` all show similar predictive performance. This indicates that while transformations are crucial for stabilizing linear models, they do not inherently create a stronger signal than the raw variable. For a linear model, it would be beneficial to select one of these transformed versions (e.g., yeojohnson_e) instead of the raw e. For a tree-based model, using the original feature e is sufficient, as trees can handle the skewness.

- *Cyclic Time Features*: The engineered features ``hour_sin`` and ``hour_cos``, often suggested in literature for temporal data, demonstrated very weak predictive power with AUCs of 0.505 and 0.501 respectively. They are not strong predictors in this dataset and will not be considered for further analysis.

- *PCA Components*: The principal components ``pca_beh_1`` (from d and m) and ``pca_amt_1`` (from e and monto) proved to be relevant. ``pca_beh_1`` had a KS of 0.166 and an AUC of 0.615, while ``pca_amt_1`` had a KS of 0.101 and an AUC of 0.561. These components successfully capture variance and could be valuable for a linear model by reducing multicollinearity.

- *Score Variable*: The score feature was highly correlated with fraud, showing a strong AUC of 0.669. This is likely a pre-calculated risk score from another model, and its high predictive power confirms its utility as a key feature.

- *General Feature Impact*: Based on their absolute AUC values, the ascending order of predictive impact for the original continuous variables is: ``c`` (0.511), ``h`` (0.519), ``b`` (0.562), ``e`` (0.562), ``m`` (0.592), ``d`` (0.615), ``l`` (0.621), and ``f`` (0.622).

Phase 02: Feature Engineering & Target Analysis

Target Characterization - General Results – Continuos variables

- *Negative Correlation of Feature d*: The feature d has a notable negative Point-Biserial correlation (r_pb = -0.078) and a respectable absolute AUC of 0.615. This indicates that as the value of d increases, the probability of fraud tends to decrease, making it a valuable risk-mitigating indicator.

- *Feature f as the Strongest Original Predictor*: Among the original, non-score variables, feature f stands out with the highest predictive power, showing an absolute AUC of 0.622 and a KS statistic of 0.11. This makes it the most impactful raw feature for separating fraud from non-fraud.

- *Weakness of Feature c*: With an AUC of just 0.511, feature `c` is barely better than a random guess. It has the lowest predictive power of all continuous variables and is a strong candidate for removal from any model to reduce noise.

- *Limited Value of Simple Interactions*: The engineered features `monto_per_e` (ratio) and `interaction_monto_e` (multiplication) did not provide a significant predictive lift over the original e feature, both having an AUC of around 0.56. This suggests these specific simple interactions do not capture new, valuable information.

- *Conclusion*: For a tree-based model, we can keep all original features as they are robust to skewness and correlation. For a linear regression model, it would be best to use transformed versions of these features (e.g., log_f, yeojohnson_e) and the PCA components to create a more stable and predictive model. The final selection of transformations should be evaluated during the modeling phase.

Phase 02: Feature Engineering & Target Analysis

Target Characterization - General Results – Categorical variables

- *Temporal Features (`is_weekend`, `weekday`):* These features were not found in the final statistics file. However, based on the very low predictive power of the continuous time features (`hour_sin`, `hour_cos`), it is likely that simple weekend or weekday flags also have little to no predictive impact on fraud.

- *Zero-Value Indicators:* The `e_is_zero` flag is not a relevant predictor, with an Information Value (IV) of only 0.012 ("Useless"). In contrast, the flags for zero values in other features, such as `f_is_zero` (IV=0.049) and `m_is_zero` (IV=0.048), show a weak but more noticeable predictive signal.

- *Missing Value Indicators:* The `b_is_missing` and `c_is_missing` flags were not relevant, both showing a negligible IV of 0.007. However, `o_is_missing` is one of the most powerful predictors in the entire dataset, with a very strong IV of 0.464. This indicates that the absence of a value in feature `o` is highly correlated with fraud.

- *Binned Continuous Variables:* Even though the original continuous variable `c` was one of the worst predictors (AUC=0.511), its binned version, `bin_c`, demonstrates a moderate predictive impact with an IV of 0.103 ("Medium"). This shows that binning can successfully capture non-linear relationships that the raw variable misses. This is a key insight: for linear models, binning can unlock predictive power by capturing non-linear patterns that the raw continuous variable misses. This strategy should be considered for other continuous variables as well.

- *Rare Grouping:* Grouping rare categories for features `a` and `j` did not significantly change their predictive power compared to their original forms. For instance, the original `a` has an IV of 0.066, and `a_rare_grouped` has a nearly identical IV. This suggests that for these variables, the rare categories themselves do not carry a distinct fraud signal.

Phase 02: Feature Engineering & Target Analysis

Target Characterization - General Results – Categorical variables

- *The Power of Binary Flags:* The features `n` and `p` are simple binary categorical variables, yet they are some of the strongest predictors. `n` has a "Strong" IV of 0.361 and `p` has a "Medium" IV of 0.246. This highlights that simple, pre-existing flags in the data can be more powerful than complex engineered features. They should be prioritized for inclusion in any model.

- *`o_is_missing` is a "Super-Predictor":* With an IV of 0.464, the `o_is_missing` flag is on the cusp of being "suspiciously" high. This means the absence of data in column `o` is a very strong indicator of fraud. While extremely valuable, it's worth double-checking that this isn't a form of data leakage (i.e., a variable that would not be available at the time of prediction in a real-world scenario).

- *Geographic and High-Cardinality Features are Weak:* The geographic feature `g` (IV=0.069) and the high-cardinality feature `j` (IV=0.082) are both classified as "Weak" predictors. While they show a statistically significant relationship to fraud, their predictive power is limited. They are unlikely to be primary drivers in a model but could provide marginal gains.

Phase 03: Model Selection & Profit Optimization

Business Objective & Financial Optimization

Our primary goal is to maximize the company's profit by effectively avoiding fraud. Currently, we face a baseline fraud rate (**5%**), which results in a significant financial loss **(-$ 547,271.12**).

In a naive baseline model where we simply approve all transactions, **we absorb this full loss**. Our objective is to build a model that decreases these fraudulent transactions and improves our bottom line.

According with previous explanation, we define the Total Profit as the sum of the financial outcomes of each prediction:

$$M = -(1.0\ x\ FN) - (0.25\ x\ FP)$$

This will be a metric used to optimize model during the modeling.

Phase 03: Model Selection & Profit Optimization

XGBoost

We use a **boosted-tree model** because boosting turns weak learners into a strong classifier and focuses training on harder (misclassified) examples; XGBoost is a widely used GBM variant for classification/ranking tasks.

In fraud settings, reaction time and prioritization by **value and risk** matter, and false positives can create customer friction—so we need a model that ranks risk well and supports thresholding to control FP/FN trade-offs.

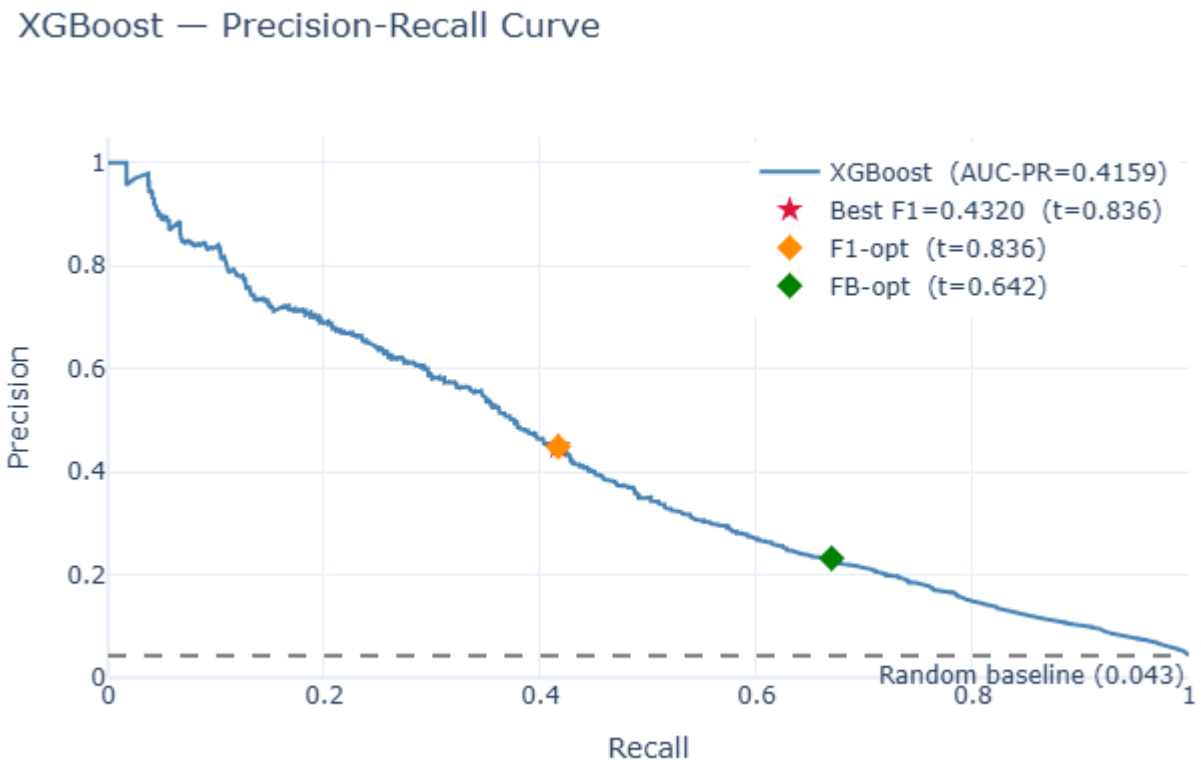
Metrics To Evaluate

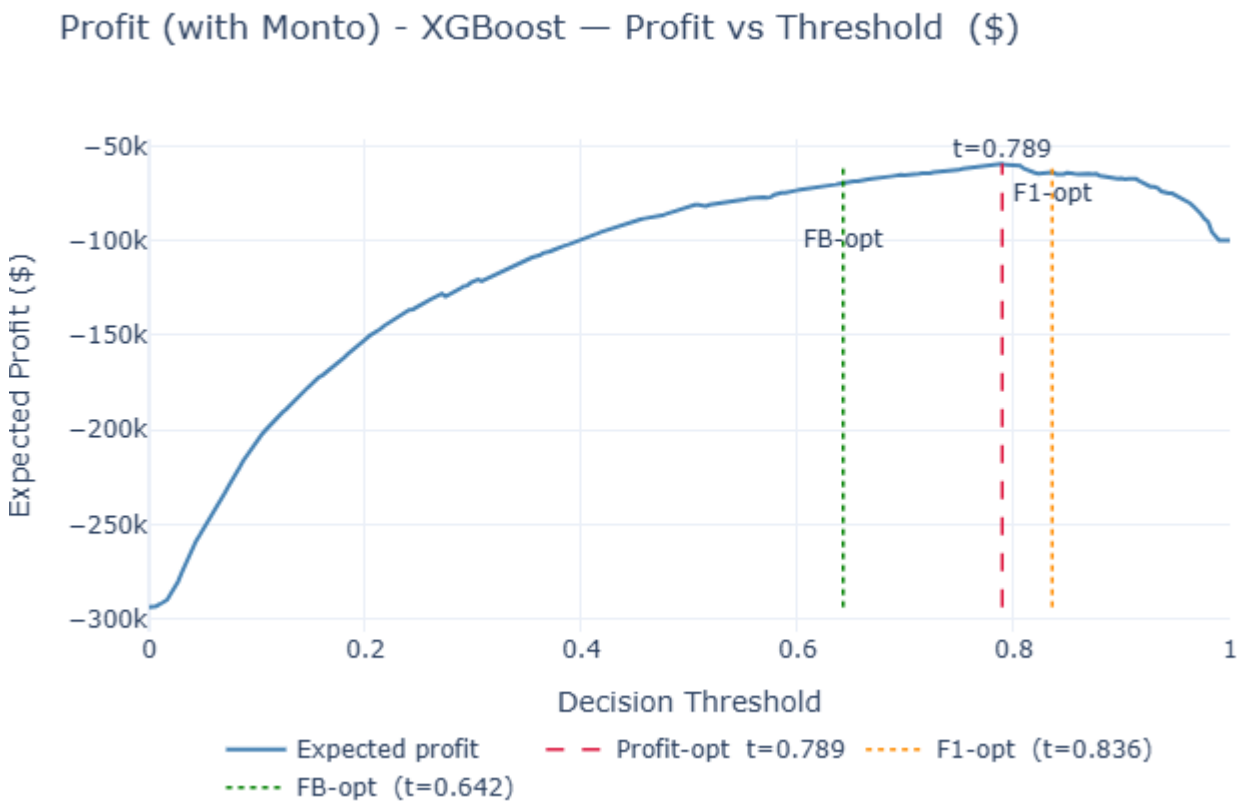
- Expected profit: profit computed from the model decisions using the cost structure (+0.25 gain for legitimate approvals, −1.00 loss for approved fraud).
- Precision: proportion of predicted fraud transactions that are actually fraud.
- Recall: proportion of total fraud transactions correctly detected by the model.
- Profit-optimal: threshold that maximizes the expected business profit.
- F1-optimal: threshold that maximizes the F1 score (balance between precision and recall).
- AUC-PR: area under the Precision–Recall curve, measuring discrimination for the positive class under imbalance.
- Fβ-score: generalized F-score that gives more weight to recall when $\beta > 1$ ($\beta=2$, in this case)
- Diff profit: difference in expected profit between the model strategy and the baseline (block-none).

Phase 03: Model Selection & Profit Optimization

Model Evaluation

We can see in the graph that when the threshold is 0 (all transactions blocked) the expected profit is −$293k, and when the threshold is 1 (all transactions approved) the expected profit is −$99k; using the optimal threshold (0.789) reduces the loss to −$59k, generating an improvement of about **$40,398** compared to the block-none strategy.

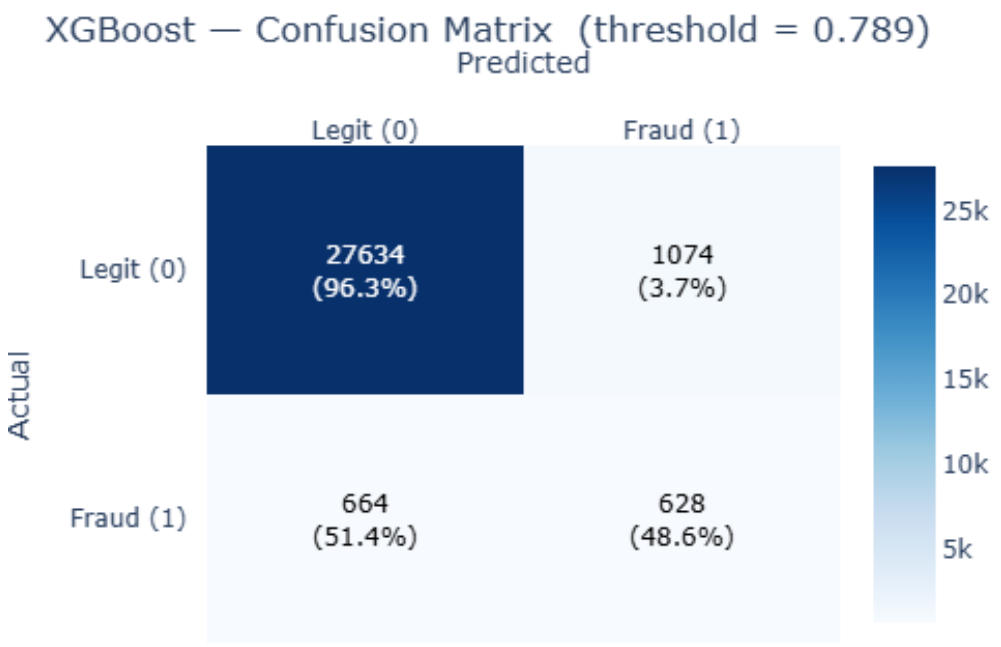




Phase 03: Model Selection & Profit Optimization

Model Evaluation:

Although precision on fraud (36.9%) reflects the challenge of a 4.3% fraud base rate, the model still detects 48.6% of fraud cases while incorrectly blocking only 3.7% of legitimate transactions.




```
Profit-block-all  t = 0.0000  →  profit = -293,509.88
Profit-block-none  t = 1.0000  →  profit = -99,824.38
Profit-optimal  t = 0.7893  →  profit = -59,426.09
Diff profit vs block-none: 40,398.29

F1-optimal      t = 0.8356  →  F1     = 0.4320
F2-optimal      t = 0.6421  →  F2     = 0.4856
AUC-PR = 0.4159   ROC-AUC = 0.8819

XGBoost — Classification Report  (threshold = 0.7893)
──────────────────────────────────────────────────────────────
              precision    recall  f1-score   support

           0     0.9765    0.9626    0.9695     28708
           1     0.3690    0.4861    0.4195      1292

    accuracy                         0.9421     30000
   macro avg     0.6728    0.7243    0.6945     30000
weighted avg     0.9504    0.9421    0.9458     30000
```

Phase 03: Model Selection & Profit Optimization

Highlights About the model:

Summary

- **Recovers ~80% of baseline losses**: reducing expected fraud loss from -$99,824.38 (no fraud detected) down to -$59,426 — a recovery of $40,398.29 on the test set.
- **Catches 48.6% of fraud cases** while **only blocking 3.7% of legitimate** transactions keeping customer friction low.
- **Precision on fraud (36.9%)** reflects the inherent difficulty of the task at a 4.3% base rate
- **Key insight**: "profit-optimal" does not mean profitable; it means least bad given your function and model.

Limitation

- **Recall gap (51.4% missed fraud):**Over half of fraud cases are not flagged at the profit-optimal threshold.
- **AUC-PR = 0.42** leaves room for improvement.

Aditional Quesitons

Question 03

3. What steps can I follow to try to ensure that the model's performance in the laboratory is similar to its performance in production?

To ensure that a model's performance in the experimental laboratory is similar to its performance in production, the most critical step is to strictly use the **exact same data processing code and transformations** across both environments. Based on my experience, here are the practical steps to achieve this:

- Unify the Processing Pipeline: In Phase 02 of my project, the initial approach used statistical functions calculated across the complete dataset at once. While fine for a notebook, this was not practical for a production environment where transactions arrive one by one. Wrapping the logic in a scikit-learn pipeline fits much better, because the exact same modular transformations (fitted on training data) can be saved and seamlessly deployed to apply the exact same logic in production.

- Design for the Correct Constraint (Throughput vs. Latency): We must remember the architectural trade-offs between environments. Research and training prioritize *throughput* (processing massive batches of historical data efficiently). Real-time production, however, prioritizes *latency* (returning a single fraud prediction in milliseconds). We must consider these scenarios and configure our deployment to respect strict production SLAs (Service Level Agreements).

- Conduct Stress Testing on Edge Datasets: Before deployment, the model must be stress-tested against different, high-volatility datasets—such as historical Black Friday traffic—to see how it handles extreme distributions.

- Monitor for Training-Serving Skew: Finally, we must proactively create metrics to measure the divergence between the data the model was trained on and the data it sees in production. Implementing statistical checks like the Population Stability Index (PSI) and Kolmogorov-Smirnov (KS) tests allows us to continuously monitor and alert on this skew.

Question 03

4. Assuming that the predictive performance in production is very different from what was expected, what do you believe are the most likely causes?

These are the principal:

- Training-Serving Skew: This is the #1 culprit for immediate degradation. An upstream ETL pipeline broke, a feature was calculated slightly differently in production, or data leakage in the lab gave the model access to information it doesn't actually have at inference time.
- Concept Drift (Fraud Evolution): Fraudsters actively adapt to your defenses. The relationship between the features and the target has fundamentally changed because the adversaries changed their tactics.
- Feature Drift: The statistical distribution of incoming data has shifted. This can be caused by marketing launching a campaign in a new region, changing the baseline "mix" of customers.
- Upstream Business Policy Changes: A rule-based system that filters transactions before they reach your model was altered, completely changing the population of data your model is scoring.
- Label Delay (Maturity Skew): In fraud detection, chargebacks can take 30 to 90 days to arrive. Short-term production metrics might look artificially good or bad simply because the true labels haven't matured yet.

Question 03

5. What steps should be followed to put the new model into production?

- Package and version the model, data schema, and feature transformations.
- Implement a production pipeline ensuring consistent feature generation.
- Deploy using controlled rollout strategies (shadow mode, A/B testing, or canary deployment).
- Establish monitoring for performance, drift, latency, and business KPIs.
- Define retraining and rollback procedures to maintain model performance over time.

Thank You!